

Package ‘dataimago’

August 25, 2025

Version 0.0-0.3

Date 2025-07-5

Title Ethical AI-Native Data Science Foundation Package

Description An R package containing the philosophical foundations, technical architecture, and application templates for dataimago's emancipatory data science platform. Provides versioned access to critical theory frameworks, application blueprints, and packageSkeleton functionality for creating AI-native data science applications that reconcile meaning (hermeneutics) and measurement (positivism).

Author Damian W. Betebenner [aut, cre] (<<https://orcid.org/0000-0003-0476-5599>>)

Maintainer Damian W. Betebenner <dbetebenner@gmail.com>

Depends R (>= 4.0)

Imports

jsonlite (>= 1.9.0), yaml (>= 2.3.0), fs (>= 1.6.0), usethis (>= 2.2.0), rlang (>= 1.1.0), crayon, glue (>= 1.7.0), whisker (>= 0.4.0), Rd2md, tools, processx (>= 3.8.0), digest (>= 0.6.0)

Suggests knitr, rmarkdown, testthat (>= 3.2.0), quarto (>= 1.4.0), shiny (>= 1.8.0), plumber (>= 1.2.0)

URL <https://dataimago.github.io/dataimago/>

BugReports <https://github.com/dataimago/dataimago/issues/>

License MIT + file LICENSE

LazyData Yes

Encoding UTF-8

Roxygen list(markdown = TRUE)

RoxygenNote 7.3.2

Config/testthat/edition 3

Contents

dataimago-package	2
add_function_documentation_sections	4
build_design_system	4
convert_rd_files_to_qmd	7
create_quarto_documentation	7
create_ui_workspace	9
extract_desc_field	11

format_description_fields 11

generate_api_reference_qmd 12

generate_cdn_assets 12

parse_description_file 13

post_process_md_to_qmd 14

update_dataimago_assets 14

update_quarto_extension 15

Index 17

dataimago-package	<i>dataimago: Ethical AI-Native Data Science Foundation Package</i>
-------------------	---

Description

The dataimago package contains the philosophical foundations, technical architecture, and application templates for dataimago’s emancipatory data science platform. It provides versioned access to critical theory frameworks, application blueprints, and packageSkeleton functionality for creating AI-native data science applications that reconcile meaning (hermeneutics) and measurement (positivism).

Details

Package Philosophy:

dataimago represents a fundamental inversion: instead of embedding culture in AI, we **embed AI in culture** for emancipatory ends. This approach transforms AI from an instrument of domination into a force for human flourishing.

Core Components:

Foundation Documents (inst/dataimago/):

- **01_Foundations/**: Mission, vision, philosophy, governance documents
- **02_Manifesto/**: Critical theory content and paradigm contrasts
- **03_Application_Architecture/**: Technical blueprints and best practices
- **04_dataimago_Content/**: Design assets and visual identity

Documentation Generation:

- [create_quarto_documentation](#): Convert R packages to ethical AI documentation websites
- Automated integration of philosophical context into technical docs
- dataimago branding and visual identity application
- MCP-compatible structured outputs for AI agent integration

Planned Functionality:

- Foundation access functions for programmatic content retrieval
- Application skeleton generation with embedded ethical frameworks
- Flow-based performance management tools
- NextJS + R integration templates

For AI Agents and MCP Integration:

This package is explicitly designed for AI agent integration:

Structured Metadata:

- Comprehensive roxygen2 documentation with philosophical context
- Consistent parameter naming and return value structures
- Machine-readable ethical annotations in generated documentation
- JSON-compatible outputs where appropriate

AI Context Files:

- `inst/CLAUDE.md`: Persistent context for AI interactions
- `inst/AGENT_INDEX.md`: Agent coordination and capabilities
- Foundation documents provide philosophical grounding for AI reasoning

MCP Tool Compatibility:

All exported functions include:

- Detailed parameter descriptions with types and constraints
- Clear return value specifications
- Comprehensive examples showing typical usage patterns
- Error conditions and handling guidance
- Philosophical context explaining the "why" behind each function

Code-as-Philosophy Commitments:

- **Hermeneutic Transparency:** All modeling assumptions documented and interpretable
- **Positivist Rigor:** Empirical operations robust, efficient, and replicable
- **Ethical Reflexivity:** Tools include affordances for users to interrogate usage
- **Iterative Design:** Each commit is dialectical step toward future aspirations

Getting Started:

Load the package and generate documentation:

```
library(dataimago)
create_quarto_documentation()
```

Access foundation documents (planned):

```
get_foundation_document("mission")
get_manifesto_content("critical_theory")
```

Generate applications (planned):

```
create_dataimago_app(name = "my-project", type = "nextjs")
bootstrap_emancipatory_framework(path = "my-project")
```

Package Metadata

- **Version:** 0.0-0.3 (using dataimago semantic versioning)
- **License:** MIT
- **Maintainer:** Damian W. Betebenner dbetebenner@nciea.org
- **Website:** <https://dataimago.github.io/dataimago/>
- **Repository:** <https://github.com/dataimago/dataimago/>

Citation

To cite dataimago in publications:

```
citation("dataimago")
```

Philosophy in Practice

"Would this object help an AI understand, improve, or operationalize emancipatory thinking? If yes, it belongs." - dataimago development philosophy

Author(s)

Maintainer: Damian W. Betebenner <dbetebenner@gmail.com> ([ORCID](#))

See Also

Useful links:

- <https://dataimago.github.io/dataimago/>
- Report bugs at <https://github.com/dataimago/dataimago/issues/>

add_function_documentation_sections

Add Function Documentation Sections with Exported/Non-exported Structure

Description

Add Function Documentation Sections with Exported/Non-exported Structure

Usage

```
add_function_documentation_sections(rd_content)
```

Arguments

rd_content List of converted .Rd content

Value

Character vector with structured function documentation

build_design_system *Build Design System Assets*

Description

Master function that orchestrates the entire CSS compilation and distribution pipeline. This function wraps Node.js tooling (pnpm/npm) in R to maintain R-first development workflow.

Usage

```
build_design_system(
  force_rebuild = FALSE,
  include_sri = TRUE,
  update_extension = TRUE,
  verbose = TRUE
)
```

Arguments

force_rebuild	Logical. Force rebuild even if assets appear up-to-date. Default: FALSE
include_sri	Logical. Generate SRI hashes for CDN distribution. Default: TRUE
update_extension	Logical. Update Quarto extension with compiled assets. Default: TRUE
verbose	Logical. Print detailed progress information. Default: TRUE

Details

This function implements dataimago's R-first philosophy by wrapping Node.js build tools in well-documented R functions. The build process includes:

System Requirements:

- Node.js 18+ and pnpm installed globally
- Write permissions to package directories
- OpenSSL for SRI hash generation (usually pre-installed on macOS/Linux)

Build Process:

1. **Node.js Dependency Check:** Verifies pnpm is available via which pnpm
2. **Dependency Installation:** Runs pnpm install in ui/ directory to install Style Dictionary, Sass, etc.
3. **SCSS Compilation:** Executes pnpm run build to convert design tokens and SCSS to minified CSS
4. **Asset Distribution:** Copies built dataimago.min.css to multiple locations:
 - quarto_website/_extensions/dataimago/ai-native/assets/css/ for Quarto extension distribution
 - inst/quarto-assets/ for R package CDN distribution
 - quarto_website/assets/css/ for local website development
5. **SRI Hash Generation:** Uses openssl dgst -sha384 -binary | openssl base64 -A for subresource integrity
6. **Build Manifest:** Creates JSON metadata with paths, versions, checksums, and build timestamp

Design Token Philosophy: The ui/ workspace uses Style Dictionary to convert semantic design tokens (color.json, typography.json, spacing.json) into CSS custom properties. This ensures design consistency across Quarto websites, R Shiny apps, and future Next.js applications while maintaining the ethical AI principles embedded in dataimago's visual identity.

Error Handling:

- Validates Node.js/pnpm availability before attempting build
- Checks for required ui/ directory structure
- Provides clear error messages with suggested remediation
- Rolls back partial builds on failure

Value

List containing build results:

- **success:** Logical indicating overall build success
- **assets:** Character vector of generated asset paths
- **sri_hashes:** Named list of SRI hashes for each CSS file
- **build_time:** POSIXct timestamp of build completion
- **metadata:** List containing build configuration and system information
- **errors:** Character vector of any error messages (empty if successful)

MCP Tool Integration

This function is designed for Model Context Protocol (MCP) integration, providing structured output that AI agents can parse and reason about:

- **Deterministic Output:** Same inputs always produce same outputs
- **Machine Readable:** JSON-compatible result structure
- **Error Context:** Detailed error messages for debugging
- **Build Metadata:** Complete provenance information for reproducibility

Philosophical Context

This function embodies dataimago's principle of "R as source of truth" while embracing modern web development practices. Rather than forcing R developers to learn Node.js toolchains, it wraps industry-standard CSS preprocessing in familiar R function calls with comprehensive documentation of every system interaction.

See Also

[update_quarto_extension](#) for extension asset management, [generate_cdn_assets](#) for CDN preparation, [create_ui_workspace](#) for UI workspace initialization

Examples

```
## Not run:
# Basic build - compile all assets
result <- build_design_system()

# Check if build succeeded
if (result$success) {
  cat("Success: Built assets:", paste(result$assets, collapse = ", "), "\n")
  cat("SRI hashes:", length(result$sri_hashes), "generated\n")
} else {
  cat("ERROR: Build failed:", paste(result$errors, collapse = "; "), "\n")
}

# Force rebuild with verbose output (useful for debugging)
result <- build_design_system(force_rebuild = TRUE, verbose = TRUE)

# Build for CDN distribution only (skip local copying)
result <- build_design_system(
  include_sri = TRUE,
  update_extension = FALSE,
```

```

    verbose = FALSE
  )

  # Inspect build metadata
  str(result$metadata)

  ## End(Not run)

```

```
convert_rd_files_to_qmd
```

Convert .Rd Files to QMD Format using Rd2md

Description

Convert .Rd Files to QMD Format using Rd2md

Usage

```
convert_rd_files_to_qmd(rd_files)
```

Arguments

`rd_files` Character vector of paths to .Rd files

Value

List containing converted documentation for each function

```
create_quarto_documentation
```

Generate Quarto Documentation from R Package

Description

Converts R package documentation (.Rd files) and metadata (DESCRIPTION) into Quarto-compatible .qmd files for website generation. Integrates dataimago branding and philosophical foundations.

Usage

```

create_quarto_documentation(
  package_path = ".",
  output_path = "quarto_website",
  include_description = TRUE,
  include_foundation_links = TRUE,
  template = "dataimago"
)

```

Arguments

package_path	Character. Path to the R package root directory. Must contain DESCRIPTION file and man/ directory with .Rd files. Default: "." (current directory)
output_path	Character. Path to quarto website directory where .qmd files will be generated. Directory will be created if it doesn't exist. Default: "quarto_website"
include_description	Logical. Whether to include full DESCRIPTION file content in the generated API reference. Useful for package metadata display. Default: TRUE
include_foundation_links	Logical. Whether to add automatic links to dataimago philosophical foundation documents in the generated documentation. Default: TRUE
template	Character. Documentation template to use for styling and structure. Currently supports "dataimago" template with ethical AI branding. Default: "dataimago"

Details

This function implements dataimago's philosophy of "code-as-philosophy" by:

- **Hermeneutic Integration:** Parses the package DESCRIPTION for metadata
- **Technical Conversion:** Uses Rd2md to convert .Rd files to Quarto markdown
- **Ethical Annotation:** Adds philosophical context to every function via `post_process_md_to_qmd()`
- **Foundation Linking:** Cross-references technical docs with philosophical content
- **Visual Identity:** Copies dataimago logos and applies custom CSS styling
- **AI Compatibility:** Generates machine-readable annotations for MCP integration

The generated documentation structure includes:

- YAML frontmatter with package metadata
- Full DESCRIPTION file content (if requested)
- Links to dataimago foundation documents
- Function documentation with ethical context boxes
- dataimago branding footer on each function

For AI agents and MCP tools, this function provides structured documentation generation that embeds philosophical context into technical reference material, enabling AI systems to understand both the "what" and "why" of each function.

Value

Character (invisible). File path to the generated `api_reference.qmd` file. Side effects: Creates .qmd files in `output_path` directory and copies dataimago assets (logos, etc.) to `assets/img/` subdirectory.

MCP Tool Integration

This function is designed for Model Context Protocol (MCP) integration:

- **Input Validation:** Checks for required package structure
- **Structured Output:** Generates consistent .qmd file format
- **Error Handling:** Provides clear error messages for debugging
- **Asset Management:** Handles file copying and directory creation
- **Status Reporting:** Uses colored console output for progress tracking

Philosophical Context

This function embodies dataimago's principle of embedding AI within culture for emancipatory purposes. Rather than treating documentation as mere technical reference, it creates living documents that connect computational tools to their ethical foundations and societal purposes.

See Also

[parse_description_file](#) for DESCRIPTION parsing, [convert_rd_files_to_qmd](#) for .Rd conversion, [update_dataimago_assets](#) for asset management

Examples

```
## Not run:
# Basic usage - generate documentation for current package
create_quarto_documentation()

# Generate for specific package with custom output location
create_quarto_documentation(
  package_path = "/path/to/my/package",
  output_path = "docs/website"
)

# Generate minimal documentation without foundation links
create_quarto_documentation(
  include_foundation_links = FALSE,
  include_description = FALSE
)

# For MCP/AI usage - capture output path
doc_path <- create_quarto_documentation()
cat("Generated documentation at:", doc_path)

## End(Not run)
```

create_ui_workspace	Create UI Workspace for Design System
---------------------	---------------------------------------

Description

Sets up the Node.js workspace structure and configuration files needed for CSS compilation and design token processing. This function creates the complete ui/ directory structure with package.json, build scripts, and source files.

Usage

```
create_ui_workspace(force_overwrite = FALSE, verbose = TRUE)
```

Arguments

force_overwrite	Logical. Overwrite existing ui/ directory if it exists. Default: FALSE
verbose	Logical. Print detailed progress information. Default: TRUE

Details

Created Directory Structure:

```

ui/
+-- package.json      # Node.js dependencies (Style Dictionary, Sass, etc.)
+-- build.js          # Custom build script for compilation
+-- src/
|   +-- tokens/
|   |   +-- colors.json    # dataimago color palette
|   |   +-- typography.json # Font definitions and scales
|   |   +-- spacing.json   # Spacing scale and dimensions
|   +-- styles/
|       +-- base.scss      # Base styles using design tokens
|       +-- components.scss # UI component styles
|       +-- utilities.scss  # Utility classes
+-- dist/                # Build output directory (created by build process)

```

Design Token Philosophy: The design tokens follow dataimago's ethical AI principles:

- **Semantic Naming:** Colors like primary, accent, ethical-highlight
- **Accessibility First:** WCAG AA contrast ratios and reduced motion support
- **Cultural Sensitivity:** Avoid culturally-biased color assumptions
- **Future-Proof:** JSON structure enables multiple output formats

Build Tool Configuration:

- **Style Dictionary:** Converts design tokens to CSS custom properties
- **Sass/SCSS:** Compiles component styles with token integration
- **PostCSS:** Adds vendor prefixes and optimizations
- **CSSnano:** Minification for production distribution

Value

List containing setup results:

- success: Logical indicating setup success
- created_files: Character vector of files created
- errors: Character vector of any error messages

Examples

```

## Not run:
# Create new ui workspace
result <- create_ui_workspace()

# Force recreate existing workspace
result <- create_ui_workspace(force_overwrite = TRUE)

# Check what was created
if (result$success) {
  cat("Created files:", paste(result$created_files, collapse = "\n "))
}

```

```
## End(Not run)
```

extract_desc_field	<i>Extract Field from DESCRIPTION Lines</i>
--------------------	---

Description

Extract Field from DESCRIPTION Lines

Usage

```
extract_desc_field(desc_lines, field_name)
```

Arguments

desc_lines	Character vector of DESCRIPTION file lines
field_name	Name of field to extract

Value

Character string with field value or NULL if not found

format_description_fields	<i>Format DESCRIPTION Fields for Quarto Display</i>
---------------------------	---

Description

Format DESCRIPTION Fields for Quarto Display

Usage

```
format_description_fields(desc_content)
```

Arguments

desc_content	Parsed DESCRIPTION content list
--------------	---------------------------------

Value

Character vector with formatted DESCRIPTION fields

```
generate_api_reference_qmd
```

Generate API Reference QMD Content

Description

Generate API Reference QMD Content

Usage

```
generate_api_reference_qmd(
  desc_content,
  rd_content,
  include_description = TRUE,
  include_foundation_links = TRUE
)
```

Arguments

```
desc_content    Parsed DESCRIPTION content
rd_content      Converted .Rd content
include_description
                 Include DESCRIPTION section
include_foundation_links
                 Include dataimago foundation links
```

Value

Character vector with complete qmd content

```
generate_cdn_assets
```

Generate CDN Distribution Assets

Description

Prepares assets for CDN distribution with versioning, SRI hashes, and metadata. Creates jsDelivr-compatible structure in inst/quarto-assets/ for R package distribution.

Usage

```
generate_cdn_assets(verbose = TRUE)
```

Arguments

```
verbose          Logical. Print detailed progress information. Default: TRUE
```

Details

This function creates CDN-ready assets by:

1. **Copying Built Assets:** Moves `ui/dist/*.css` to `inst/quarto-assets/`
2. **Generating Metadata:** Creates `manifest.json` with build info and SRI hashes
3. **Creating Usage Examples:** Generates HTML snippets showing CDN usage
4. **Validating File Integrity:** Ensures all copied files are complete and uncorrupted

CDN Distribution Strategy: Files in `inst/quarto-assets/` become available via jsDelivr once the package is tagged and released on GitHub:

`https://cdn.jsdelivr.net/gh/dataimago/dataimago-rpkg@v0.1.0/inst/quarto-assets/dataimago.min.css`

SRI (Subresource Integrity) Hashes: Each CSS file gets a SHA-384 hash for security:

```
<link rel="stylesheet"
      href="https://cdn.jsdelivr.net/gh/.../dataimago.min.css"
      integrity="sha384-..."
      crossorigin="anonymous">
```

Generated Files:

- `inst/quarto-assets/dataimago.min.css` - Main compiled CSS
- `inst/quarto-assets/tokens.css` - CSS custom properties (if built)
- `inst/quarto-assets/manifest.json` - Build metadata and SRI hashes
- `inst/quarto-assets/README.md` - Usage instructions and examples

Value

List containing CDN preparation results:

- `success`: Logical indicating preparation success
- `assets`: Character vector of CDN-ready asset paths
- `errors`: Character vector of any error messages

`parse_description_file`

Parse DESCRIPTION File

Description

Parse DESCRIPTION File

Usage

```
parse_description_file(desc_file)
```

Arguments

`desc_file` Path to DESCRIPTION file

Value

List containing parsed DESCRIPTION fields

```
post_process_md_to_qmd
```

Post-Process Markdown to QMD Format with dataimago Customizations

Description

Post-Process Markdown to QMD Format with dataimago Customizations

Usage

```
post_process_md_to_qmd(md_content, func_name)
```

Arguments

md_content	Character vector of markdown content from Rd2md
func_name	Function name for customization

Value

Character vector with qmd-formatted content

```
update_dataimago_assets
```

Update dataimago Assets in Quarto Website

Description

Update dataimago Assets in Quarto Website

Usage

```
update_dataimago_assets(output_path, package_path)
```

Arguments

output_path	Path to quarto website directory
package_path	Path to package root

update_quarto_extension

Update Quarto Extension Assets

Description

Copies compiled CSS and other assets from `ui/dist/` into the Quarto extension structure. Maintains proper `extension.yml` configuration and ensures asset linking works correctly.

Usage

```
update_quarto_extension(verbose = TRUE)
```

Arguments

`verbose` Logical. Print detailed progress information. Default: TRUE

Details

This function performs the following operations:

1. **Validates Extension Structure:** Checks that `quarto_website/_extensions/dataimago/ai-native/` exists
2. **Copies Main CSS:** Copies `ui/dist/dataimago.min.css` to extension assets
3. **Updates Asset References:** Ensures `extension.yml` references the correct CSS files
4. **Preserves Extension Config:** Maintains existing Lua filters, shortcodes, etc.

Extension Asset Mapping:

- `ui/dist/dataimago.min.css` -> `quarto_website/_extensions/dataimago/ai-native/assets/css/dataimago.min.css`
- `ui/dist/tokens.css` -> `quarto_website/_extensions/dataimago/ai-native/assets/css/tokens.css` (if exists)
- Source maps and development files are excluded from extension distribution

File Operations: All file copying uses R's `fs::file_copy()` with overwrite protection and atomic operations to prevent corrupted assets during development. The function will not overwrite extension configuration files (like `_extension.yml`) unless they become corrupted.

Value

List containing update results:

- `success`: Logical indicating update success
- `updated_files`: Character vector of files updated in extension
- `errors`: Character vector of any error messages

Examples

```
## Not run:
# Update extension after building CSS
build_design_system()
result <- update_quarto_extension()

# Check what was updated
if (result$success) {
  cat("Updated files:", paste(result$updated_files, collapse = "\n "))
}

## End(Not run)
```

Index

- * **build-tools**
 - build_design_system, [4](#)
 - * **critical-theory**
 - dataimago-package, [2](#)
 - * **css**
 - build_design_system, [4](#)
 - * **dataimago emancipatory-ai**
 - MCP-compatible**
 - foundation-package**
 - dataimago-package, [2](#)
 - * **dataimago ethical-documentation**
 - MCP-compatible**
 - create_quarto_documentation, [7](#)
 - * **dataimago r-first-development**
 - MCP-compatible**
 - build_design_system, [4](#)
 - * **design-system**
 - build_design_system, [4](#)
 - * **documentation-generation**
 - dataimago-package, [2](#)
 - * **documentation**
 - create_quarto_documentation, [7](#)
 - * **ethical-ai**
 - build_design_system, [4](#)
 - create_quarto_documentation, [7](#)
 - dataimago-package, [2](#)
 - * **generation**
 - create_quarto_documentation, [7](#)
 - * **internal**
 - add_function_documentation_sections, [4](#)
 - convert_rd_files_to_qmd, [7](#)
 - extract_desc_field, [11](#)
 - format_description_fields, [11](#)
 - generate_api_reference_qmd, [12](#)
 - parse_description_file, [13](#)
 - post_process_md_to_qmd, [14](#)
 - update_dataimago_assets, [14](#)
 - * **package**
 - dataimago-package, [2](#)
 - * **quarto**
 - create_quarto_documentation, [7](#)
- add_function_documentation_sections, [4](#)
- build_design_system, [4](#)
- convert_rd_files_to_qmd, [7](#), [9](#)
- create_quarto_documentation, [2](#), [7](#)
- create_ui_workspace, [6](#), [9](#)
- dataimago (dataimago-package), [2](#)
- dataimago-package, [2](#)
- extract_desc_field, [11](#)
- format_description_fields, [11](#)
- generate_api_reference_qmd, [12](#)
- generate_cdn_assets, [6](#), [12](#)
- parse_description_file, [9](#), [13](#)
- post_process_md_to_qmd, [14](#)
- update_dataimago_assets, [9](#), [14](#)
- update_quarto_extension, [6](#), [15](#)